Python Programming Fundamentals

The super() Function

Programming Fundamentals Team

SETU

2026-06-12

Outline

1. What is `super()`?
2. Calling Parent `__init__`
3. Extending Parent Methods
4. When to Use `super()`
5. Multiple Levels of Inheritance
6. Method Resolution Order (MRO)
7. Summary

Outline

1. What is super()?
2. Calling Parent __init__
3. Extending Parent Methods
4. When to Use super()
5. Multiple Levels of Inheritance
6. Method Resolution Order (MRO)
7. Summary

1.1 A reference to the parent class

`super()` returns a proxy object that lets you call methods defined in the **parent class** from inside the **child class**.

Most commonly used in `__init__`:

```
class Child(Parent):
    def __init__(self, parent_arg1, parent_arg2, child_arg):
        super().__init__(parent_arg1, parent_arg2)
        self.__child_attr = child_arg
```

Without `super().__init__()`, the parent's attributes are never set up!

Outline

1. What is super()?
2. Calling Parent __init__
3. Extending Parent Methods
4. When to Use super()
5. Multiple Levels of Inheritance
6. Method Resolution Order (MRO)
7. Summary

2. Calling Parent __init__

```
class Person:
    def __init__(self, name: str, email: str):
        self.__name = name
        self.__email = email

class PremiumMember(Person):
    def __init__(self, name: str, email: str,
                 subscription: str):
        super().__init__(name, email)   # sets __name, __email
        self.__subscription = subscription

pm = PremiumMember("Alice", "alice@example.com", "Gold")
print(pm.get_name())   # Alice  ← set by Person.__init__
```

Outline

1. What is super()?

2. Calling Parent __init__

3. Extending Parent Methods

4. When to Use super()

5. Multiple Levels of Inheritance

6. Method Resolution Order (MRO)

7. Summary

3.1 Adding to, not replacing

```python
class Person:
    def describe(self) -> str:
        return f"Name: {self.__name}, Email: {self.__email}"

class PremiumMember(Person):
    def describe(self) -> str:
        base = super().describe()   # get parent's text
        return f"{base}, Plan: {self.__subscription}"

pm = PremiumMember("Alice", "alice@example.com", "Gold")
print(pm.describe())
# Name: Alice, Email: alice@example.com, Plan: Gold
```

Outline

1. What is super()?
2. Calling Parent __init__
3. Extending Parent Methods
4. When to Use super()
5. Multiple Levels of Inheritance
6. Method Resolution Order (MRO)
7. Summary

4. When to Use super()

- **Always** call `super().__init__()` in a subclass `__init__` (unless the parent has no `__init__` worth calling)
- Use `super().method()` when you want to **extend** (not replace) a parent method
- Do **not** call `super()` if you want to **completely replace** the parent method

```
# EXTENDING — calls parent first, then adds
def __str__(self):
    return super().__str__() + f" [Premium]"

# REPLACING — ignores parent completely
def __str__(self):
    return f"PremiumMember: {self.get_name()}"
```

Outline

1. What is super()?
2. Calling Parent __init__
3. Extending Parent Methods
4. When to Use super()
5. Multiple Levels of Inheritance
6. Method Resolution Order (MRO)
7. Summary

5. Multiple Levels of Inheritance

```
class Person:
    def __init__(self, name): self.__name = name
    def greet(self): return f"Hi, I'm {self.__name}"

class User(Person):
    def __init__(self, name, username):
        super().__init__(name)
        self.__username = username

class AdminUser(User):
    def __init__(self, name, username, access_level):
        super().__init__(name, username)   # calls User.__init__
        self.__access_level = access_level
```

`super()` follows the **Method Resolution Order** (MRO) — Python's rule for which class to look in next.

Outline

1. What is super()?
2. Calling Parent __init__
3. Extending Parent Methods
4. When to Use super()
5. Multiple Levels of Inheritance
6. Method Resolution Order (MRO)
7. Summary

6. Method Resolution Order (MRO)

```python
class A: pass
class B(A): pass
class C(B): pass

print(C.__mro__)
# (<class 'C'>, <class 'B'>, <class 'A'>, <class 'object'>)
```

When you call a method on `C`, Python searches: `C` → `B` → `A` → `object`.

The first class in the MRO that defines the method wins.

`super()` moves to the **next** class in the MRO.

Outline

1. What is super()?
2. Calling Parent __init__
3. Extending Parent Methods
4. When to Use super()
5. Multiple Levels of Inheritance
6. Method Resolution Order (MRO)
7. Summary

7. Summary

Use	Code
Call parent `__init__`	`super().__init__(args)`
Extend a method	`base = super().method()` then add to it
Check MRO	`MyClass.__mro__`
Replace a method	Just define it in child (no `super()`)

Thanks for Watching - Any questions?